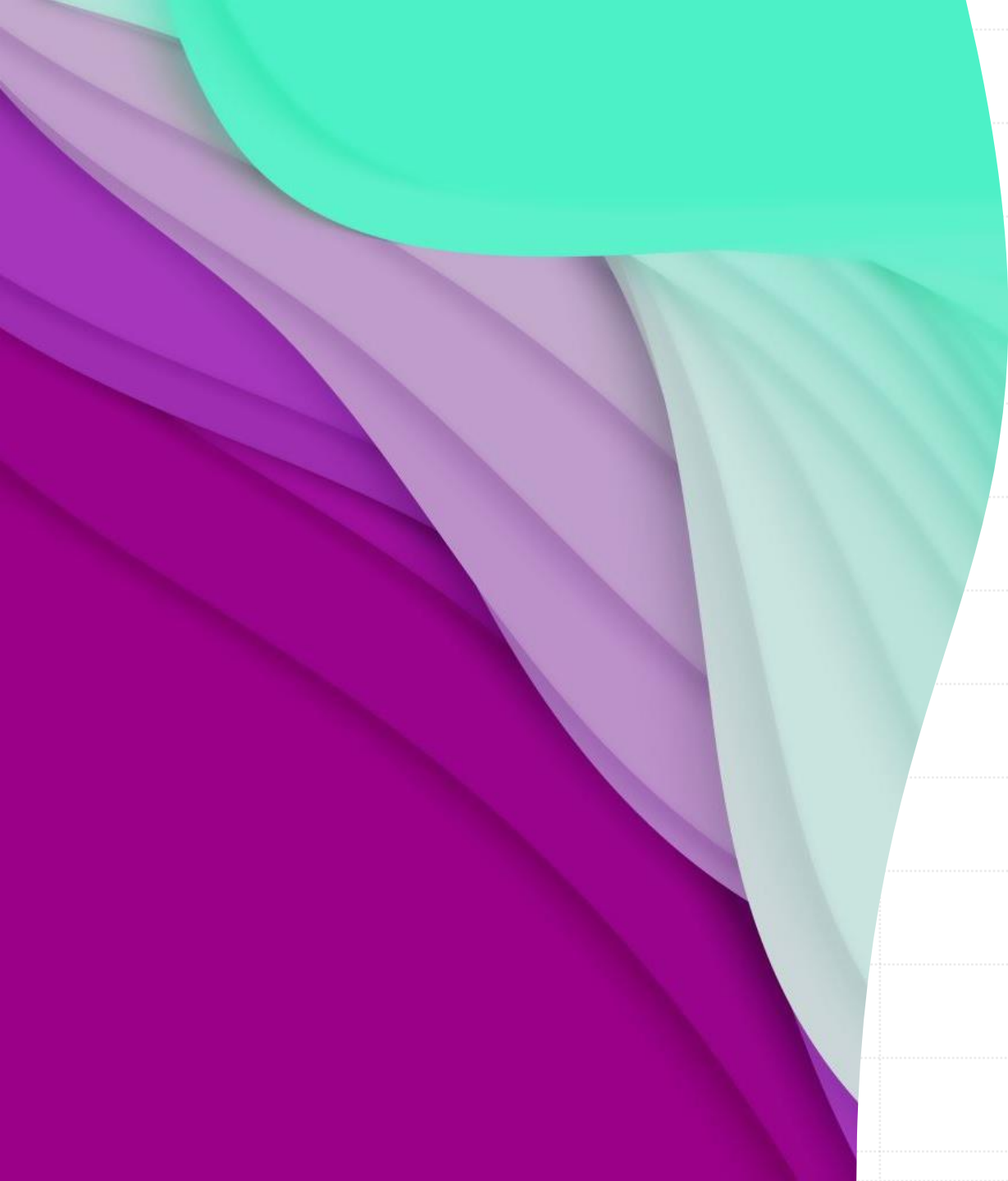




Python Programming Bootcamp – Week 3, Day 2

Lists – Advanced Operations

Atta Muhammad

attapanhyar@quest.edu.pk

+923331971110

Quaid-e-Awam University of Engineering,
Science and Technology



Learning Objectives

- Understand advanced list operations such as list comprehension, sorting, and reversing.
- Learn how to manipulate lists using slicing and built-in functions.
- Explore practical applications of lists in Python programming.



Review of Lists

- **Definition:** Lists are mutable collections of ordered items.

- **SYNTAX**



List Comprehension

- **Definition:** A concise way to create lists.

#Syntax:

```
[expression for item in iterable]
```

#Example:

```
squares = [x**2 for x in range(1, 6)]
```

```
print(squares) # Output: [1, 4, 9, 16, 25]
```



Sorting Lists

- **Definition:** Sort the elements of a list in place or return a new sorted list.

```
numbers = [4, 2, 9, 1]
numbers.sort() # Sort in ascending order
print(numbers) # Output: [1, 2, 4, 9]
#SORTED
sorted_list = sorted(numbers)
```



Reversing Lists

- **Definition:** Reverse the order of elements in a list.

```
numbers = [4, 2, 9, 1]
numbers.reverse()
print(numbers) # Output: [9, 4, 2, 1]
```



List Slicing

- **Definition:** Extract a subset of the list.

```
# SYNTAX:
```

```
|   # list[start:end]
```

```
numbers = [1, 2, 3, 4, 5]
```

```
print(numbers[1:4])  # Output: [2, 3, 4]
```



Finding the Minimum and Maximum

- Functions: **min()** and **max()**

```
numbers = [1, 4, 2, 9, 6]
print(min(numbers)) # Output: 1
print(max(numbers)) # Output: 9
```




Hands-On Exercise 1

- **Task:** Write a Python program that creates a list of even numbers from 1 to 20 using list comprehension.

Joining Lists

- **Definition:** Combine two lists into one.



```
list1 = [1, 2]
list2 = [3, 4]
combined = list1 + list2
print(combined)  # Output: [1, 2, 3, 4]
```



List Methods Overview

- Common Methods:
 - **append()**
 - **extend()**
 - **insert()**
 - **remove()**
 - **pop()**
 - **clear()**

Hands-On Exercise 2

- **Task:** Write a Python program that sorts a list of numbers, reverses the sorted list, and prints both.





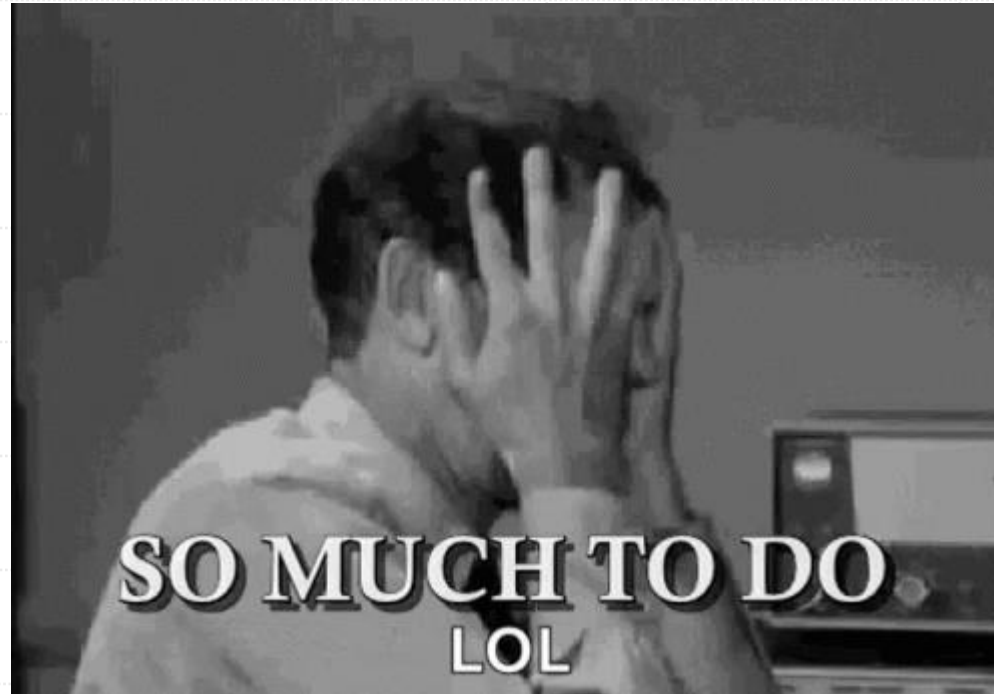
Nested Lists

- **Definition:** A list within a list.

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]  
print(matrix[1][2]) # Output: 6
```

Hands-On Group Activity

- **Task:** Create a nested list representing a matrix and calculate the sum of each row.





Advanced List Functions

- **sum()**: Returns the sum of all elements in a list.
- **any()**: Returns True if at least one element is true.
- **all()**: Returns True if all elements are true.



List Comprehension with Conditions

```
even_squares = [x**2 for x in range(1, 11) if x % 2 == 0]  
print(even_squares) # Output: [4, 16, 36, 64, 100]
```




Recap and Key Takeaways

- Lists are a powerful data structure in Python.
- You can use list comprehension to simplify code.
- List methods make it easy to manipulate data.



Q&A and Wrap-Up



MATCH THE SOLUTIONS





EXERCISE 🙋

Step 1: Use list comprehension to create a list of even numbers

```
even_numbers = [x for x in range(1, 21) if x % 2 == 0]
```

Step 2: Print the list of even numbers

```
print(even_numbers)
```

EXERCISE 🤪

```
# Step 1: Create a list of numbers
```

```
numbers = [4, 2, 9, 1, 7]
```

```
# Step 2: Sort the list in ascending order
```

```
numbers.sort()
```

```
print("Sorted list:", numbers)
```

```
# Step 3: Reverse the sorted list
```

```
numbers.reverse()
```

```
print("Reversed list:", numbers)
```

EXERCISE 🤪

Step 1: Create a nested list (matrix)

```
matrix = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]
```

Step 2: Use a for loop to calculate the sum of each row

```
for row in matrix:  
    row_sum = sum(row)  
    print(f"Sum of row {row}: {row_sum}")
```